cardiac_core — API cheatsheet

> **Canonical, maintained reference.** Every signature here is real; co-located with the code so it can't drift. **After any `cardiac_core` API change, re-run the two canaries that execute code straight out of this file** — if they break, fix this file: `tests/test_usability_fixes.py::test_cheatsheet_examples_execute` (the `# runnable-canary` block, §12) and `tests/test_video.py::test_cheatsheet_video_section_executes` (the `# runnable-video-section` block, §10).

Verified against the shipped `cardiac_core` (481 passed / 2 xfailed, 2026-07-23). CPU + `float64` by default; deterministic.

```
import cardiac_core as cc
```

1. Geometry — `cc.Grid`

```
g = cc.Grid(Nx, Ny, dx, dy=None, *, mask=None, device="cpu")   # dy defaults to dx
g.Nx, g.Ny, g.dx, g.dy        # ints / floats
g.Lx, g.Ly                    # dx*(Nx-1), dy*(Ny-1)  (cm)
g.coordinates                 # (x, y) tensors, shape (Nx, Ny), ij indexing  (x[-1,0] == Lx)
g.n_dof                       # Nx*Ny, or mask.sum()
```

`Grid.Lx = dx*(Nx-1)`, so for an exact **Lx × Ly cm** domain use `Nx = round(Lx/dx) + 1`, `Ny = round(Ly/dy) + 1`. A **2 cm × 0.5 cm strip at dx=0.01**: `cc.Grid(201, 51, 0.01)` (Lx = 0.01×200 = 2.0 cm). A **1-D cable** is `Ny=1`: `cc.Grid(201, 1, 0.01)` (the degenerate y-axis inherits `dx`).

Optional `mask` (irregular tissue) — `(Nx, Ny)` bool from a helper:

```
mask = cc.left_edge_mask(Nx, Ny, dx, width)          # left strip
mask = cc.circle_mask(Nx, Ny, dx, center=(cx,cy), radius=r)
mask = cc.rectangle_mask(Nx, Ny, dx, x0, y0, x1, y1)
mask = cc.annulus_mask(Nx, Ny, dx, center, inner_radius, outer_radius)
```

Masked-out (out-of-domain) nodes come back as **NaN** in `r.Vm` (monodomain/bidomain), so `cv/apd/lat` correctly skip them. A masked bidomain now **auto-selects** `pcg` for the elliptic solve, so the default works on a hole. **LBM** masked nodes stay at resting voltage (not NaN).

2. Conductivity — `cc.ConductivityConfig`

```
cond = cc.ConductivityConfig.isotropic(sigma, chi=1400.0, Cm=1.0)         # single effective σ
cond = cc.ConductivityConfig.bidomain(sigma_i, sigma_e, chi=1400.0, Cm=1.0)  # most physical
cond = cc.ConductivityConfig.anisotropic(sigma_l, sigma_t, fiber_angle, chi=1400.0, Cm=1.0)
cond.sigma_eff   # effective conductivity (mS/cm) — a scalar for isotropic/bidomain,
                 #   but a 3-tuple (xx, yy, xy) for anisotropic (don't float() it blindly)
cond.D_eff       # derived diffusivity = sigma_eff/(chi*Cm) (cm^2/ms) — same scalar-or-3-tuple shape
```

`fiber_angle` is in **radians** (0 = fibers along +x). `anisotropic` uses ONE global angle; per-node fiber fields are not exposed by the factories.

> ⚠️ **Units trap.** `sigma*` are **raw conductivities in mS/cm**. The diffusivity `D_eff` is DERIVED — do NOT pass a pre-divided `D` as `sigma`. Healthy human ventricle: `sigma_i=1.74, sigma_e=6.25` → `sigma_eff≈1.36` → `D_eff≈0.000972 cm²/ms` → CV ≈ 55–60 cm/s (solver/dx-dependent; the strip example below gives 59.3 at dx=0.01/CN-PCG). Lower σ = weaker coupling (fibrosis-like).

3. Stimulus — a `Stim` object (canonical), or a list of them

```
stim = cc.Stim.boundary(g, "left")                    # a border strip (left/right/top/bottom)
stim = cc.Stim.point(g, (0.5, 0.3))                   # a blob at an (x, y) cm point
stim = cc.Stim.center(g)                              # a blob at the domain centre
stim = cc.Stim.from_region(g, lambda x, y: x < 0.05)  # any callable (x,y)->bool mask / (Nx,Ny) array
stim = cc.Stim(mask)                                  # an explicit (Nx, Ny) bool mask
```

Every factory takes the usual keywords: `amplitude=-52.0` µA/µF (negative = depolarizing), `start_time`, `duration` (ms), and `bcl` / `num_pulses` for a pacing train. Pass a **list** of `Stim`s for multi-site / moving stimuli (overlaps sum on FDM/FEM; LBM overwrites). Two **modes**, inferred from the keyword:

```
cc.Stim.boundary(g, "left", amplitude=-52, bcl=1000, num_pulses=5)   # CURRENT injection (Istim)
cc.Stim.boundary(g, "left", clamp=-20, duration=50)                  # VOLTAGE clamp (hard override, all
engines)
```

A `clamp=<mV>` Stim holds V (not Istim) on its nodes — routed to `clamp_voltage`, never serialized as a current.

> **Legacy dict form** (still works, now `DeprecationWarning`):
> `{"region": lambda x,y: x<0.05, "start_time":1.0, "duration":2.0, "amplitude":-80.0, "bcl":..., "num_pulses":...}`.
> Prefer `Stim`.

4. Construct — pick the engine by calling its factory

```
sim = cc.monodomain(g, "ttp06", cond, stim)     # fast, single potential — DEFAULT choice
sim = cc.bidomain(g, "ttp06", cond, stim)       # ONLY when bath/boundary/edge effects matter
sim = cc.lbm(g, "ttp06", cond, stim, dt=0.005)  # lattice-Boltzmann (smaller dt)
```

- **Ionic models** and where each runs: | name | model | monodomain / bidomain | lbm | |------|-------|:---:|:---:| | `"ttp06"` | ten Tusscher 2006 (human ventricle, default) | ✅ | ✅ | | `"ord"` | O'Hara–Rudy 2011 (human ventricle) | ❌ **LBM only** | ✅ | | `"paci"` | Paci 2013 (**hiPSC-CM**, spontaneously active) | ✅ | ✅ | | `"phas13"` / `"mhas13"` | Paci/Mahajan hiPSC variants | ✅ | ✅ | `"ord"` on monodomain/bidomain raises (its SR-release/CaMKII concentration path isn't wired for the classical splitting) — use `cc.lbm(..., "ord", ...)`.
- **Engine rule for generation: monodomain** unless the experiment is about the surrounding bath / tissue edge / boundary loading → then **bidomain** (`boundary="bath"` adds an elliptic solve per step — noticeably slower than `"insulated"`; only use it when the bath matters).
- **LBM** CV runs **~30–47% higher** than the FDM monodomain for the same σ (different numerics) — compare LBM-to-LBM, not LBM-to-monodomain.
- **LBM wall modes** (`cc.lbm(..., lattice="d2q9", boundary=...)`): the flat top/bottom boundary-speedup family `"hbb"` / `"ncs"` / `"scs"` / `"combined"` (+`alpha` for `combined`) **requires** `lattice="d2q9"` — requesting them on the default `lattice="d2q5"` raises. The default `boundary` is lattice-aware (`neumann` on d2q5, `hbb` on d2q9); only `neumann` is valid on d2q5.
- `device="cpu"` (default) or `"cuda"`; float64 throughout.

5. Solver & `dt` — the main speed lever

```
sim = cc.monodomain(g, "ttp06", cond, stim,
                    dt=0.05,                    # ms; the biggest lever on wall-time (fewer steps)
                    diffusion_solver="crank_nicolson",  # default; UNCONDITIONALLY STABLE
                    linear_solver="pcg")        # default; "dct" is the fast direct path (below)
```

- `dt`: Crank–Nicolson (the default) is unconditionally stable, so **raise** `dt` **for speed** — 0.05–0.1 ms is usually fine for CV/APD; accuracy, not stability, is the limit. Cost is ~linear in `t_end/dt` (~1.5–3 ms/step on CPU, ~13 ms/step on GPU, roughly grid-size-independent for the default solver — long protocols are step-bound, so a bigger `dt` is the win).
- `linear_solver="dct"`: a direct O(N log N) spectral solve that matches CN exactly and is much faster than the default `"pcg"` on large grids — but ONLY on an **isotropic-uniform, full-rectangle** mesh with the default `boundary_mode="face_mirror"` / `stencil="cardinal4"` and `crank_nicolson` / `bdf1`. It inverts an idealized eigen-operator, so anything else (a mask/scar, anisotropic fibers, non-default stencil/BC, or

`bdf2`) would be silently wrong — the factory now **raises** in those cases; use `"pcg"` there. `"fft"` (periodic BC) is **not usable** via these factories (they build Neumann meshes) and raises. `"pcg"` is the robust default.

- `diffusion_solver="forward_euler"` is explicit (no linear solve) but CFL-limited: it **warns** if `dt > chi*Cm*min(dx,dy)²/(4*D_max)` and then oscillates/blows up — keep `dt` under that or stay on CN.

6. Run — eager `SimulationResult`

```
r = sim.run(t_end, save_every=1.0)              # eager: one SimulationResult
for chunk in sim.run(t_end, save_every, batch=50):  # streamed chunks (large runs)
    ...
r = sim.run(t_end, save_every, record=("Vm", "ionic_states"))  # opt in to ionic-state history
r.Vm        # (T, Nx, Ny) float64 voltage history
r.times     # (T,) ms
r.dx, r.dy  # spacing
r.phi_e     # (T,Nx,Ny) bidomain only, else None
r.ionic_states   # (T, n_states, Nx, Ny) only if record=(...,"ionic_states")
```

`record=` accepts only `"Vm"` and `"ionic_states"` — an unknown key (e.g. `"I_Kr"`) raises, it does not silently record nothing. (`"ionic_states"` is monodomain/bidomain only — LBM raises `NotImplementedError`.)

7. Measure — hooks on the result (+ `cc.<analysis>`)

```
cv  = r.cv(x1, x2, y)          # conduction velocity (cm/s) between x-indices x1<x2 at row y
apd = r.apd()                  # (Nx,Ny) APD90 map (ms); apd(repol=0.5) = APD50
lat = r.lat()                  # (Nx,Ny) local activation-time map (ms)
rst = r.restitution(ix, iy)    # (DI, APD) restitution at a node (multi-beat run)
# CANONICAL LAT: r.lat()/r.cv()/cv_between/radial_cv all use ONE activation map —
#   interpolated sub-frame crossing at -40 mV (method="interp"). Pass method="nearest",
#   threshold=-20 to reproduce the pre-2026-07-22 frame-quantized value. A first-crossing
#   LAT is invalid under reentry (a rotor re-activates nodes) — use phase_map there.
# direct analysis (same functions): cc.conduction_velocity(Vm,times,dx,x1,x2,y), cc.apd_map(Vm,times),
# cc.activation_time(Vm,times), cc.max_dvdt_time(Vm,times), cc.restitution_curve(Vm,times,ix,iy),
# cc.apd_at(Vm,times,ix,iy)
# fibrillation: cc.dominant_frequency(Vm, times, ix, iy)   # Hz at one node
# rotor tips (two steps): phase = cc.phase_map(Vm, times, t_idx); s = cc.phase_singularities(phase)
```

Aggregate / per-beat / axis hooks (P2):

```
dfm = r.df_map()               # (Nx,Ny) dominant-frequency map (Hz) — warns if freq res is coarse
cv2 = r.cv_between((i1,j1), (i2,j2))   # CV along ANY direction (not just the x axis)
rcv = r.radial_cv((ci,cj))     # (Nx,Ny) outward-CV map from a point source
apb = r.apd_per_beat(ix, iy)   # (n_beats,) APD of each beat (each bounded to its own beat)
rs  = r.restitution_slope(ix, iy)  # {max_slope, DI_star (alternans onset, ms), n}
```

- CV indices: choose `x1,x2` well inside the tissue and give the front time to reach `x2` (front ≈ 50 cm/s = 0.05 cm/ms; 1 cm ≈ 20 ms — set `t_end` accordingly). NaN CV = the front didn't reach a point (e.g. conduction block) within the run.
- `phase_singularities(phase)` returns a `(Nx-1, Ny-1)` topological-charge map; |charge| ≈ 1 marks a tip.

Named field maps — `r.fields.*` (lazy, cached; torch, on-device)

```
# Vm-based (per-frame (T,Nx,Ny[,2])). Vectors are VectorField: .x .y .magnitude .angle .components
r.fields.voltage_gradient      # ∇Vm                          (VectorField)
r.fields.voltage_flux          # D_eff·∇Vm                    (VectorField)
r.fields.source_sink           # ∇·(D_eff∇Vm) = the electrotonic source–sink map (monodomain, iso)
r.fields.electric_field        # −∇φ_e   (bidomain only)      (VectorField)
r.fields.current_flux          # −σ_e·∇φ_e (bidomain only)    (VectorField)
# LAT-based ((Nx,Ny), no time axis) — all from the canonical interp/−40 LAT, divergence-gated:
r.fields.velocity  .direction  .speed   # ∇T/|∇T|² , n̂ , 1/|∇T| (cm/s); NaN at collisions
r.fields.curvature .vorticity  .quality .mask   # ∇·n̂ , curl(v) , fit residual , gate (True=low-confidence)
# operator toolkit (grid/boundary/mask-bound) and reductions:
r.fields.derivatives.grad(f) / .div(F) / .curl(F) / .laplacian(f)
r.fields.integrals.region_integral(f, over=mask)   # ∬f dA
r.fields.integrals.net_flux(F, region=mask)        # ∮F·n = ∬div F  (divergence theorem)
r.fields.integrals.circulation(v, region=mask)     # ∮v·dl = ∬curl v (Stokes)
r.fields.integrals.winding_number(phase)           # enclosed rotor count
r.fields.integrals.conduction_time((ix,iy),(jx,jy))# ΔT (integrate slowness ∇T, NOT velocity)
r.fields.integrals.activated_area()                # (T,) depolarized area per frame
r.fields.integrals.wavefront_length(at_time=t) / .global_curvature(at_time=t)   # isochrone ∮ds / ∮kds
```

`source_sink` / `voltage_flux` are monodomain + isotropic (raise otherwise); `electric_field` / `current_flux` are bidomain-only.

Scalar EP + protocols + single-cell

```
cc.wavelength(cv_cms, refractory_ms, kind="erp")   # λ = CV·ERP /1000 (cm); kind="apd" warns
cc.di(bcl, apd)                                     # diastolic interval BCL−APD
cc.erp(grid, 'ttp06', cond, bcl=1000, n_s1=4)      # ERP via S1S2 capture bisection (RUNS sims)
sc = cc.single_cell('ttp06', celltype='EPI', pre_pace=5)   # 0-D AP; sc.V, sc.apd(0.9), sc.final_state
cc.safety_factor(r, q_thr=...)                     # (Nx,Ny) Boyle–Vigmond SF (∫source_sink/Q_thr); <1 = block
```

8. Heterogeneity — drug block, scar, conductivity (each rebuilds from t=0)

```
sim.scale_conductance("GKr", 0.5)          # 50% IKr block (dofetilide-like) -> prolongs APD
sim.set_conductivity(scar_mask, D=0.0)     # inexcitable scar; wave routes around (D=0 = zero flux)
sim.scale_conductivity(border_mask, 0.3)   # slow-conduction zone (× current D)
```

- **`scale_conductance(name, factor)`** — `name` is the ionic-model PARAMETER, not the current name. Common knobs (both TTP06 and ORd): `GNa` (INa), `GKr` (IKr), `GKs` (IKs), `GK1` (IK1), `Gto` (Ito), **`PCa` (ICaL — a permeability, NOT "GCaL")**; ORd adds `GNaL` (INaL). An unknown name raises and lists the model's conductances. `factor<1` = block, `>1` = upregulation; repeated calls compound. Global scalar only (no per-node `mask=`/Distribution). Rebuilds from t=0 on the live model, so cell type is preserved.
- **`set_conductivity(mask, D)`** — absolute RAW `D` on `mask` (like `create_cardiac_mesh`'s `D`; effective = `D/(chi*Cm)`). `D=0.0` is a scar. On a bidomain built from `ConductivityConfig` (sigma fields), only `D=0` is meaningful — a nonzero absolute `D` raises (use `scale_conductivity` instead).
- **`scale_conductivity(mask, factor)`** — multiply the region's conductivity by `factor` (works on both the D-field and bidomain-sigma representations).

Other `CardiacSimulation` helpers (`get_state`, `clamp_voltage`, `add_pacing`, `set_parameter`, `add_probe`, `compute_cv/apd`, …) are **not implemented** and raise an informative error naming the real route — do not generate against them. For state history use `record="ionic_states"`; for analysis use the `r.` hooks above.

9. Save / load a result (`.npz`)

```
cc.save_result(path, r.times, r.Vm)          # arg order: path, times, Vm  (+ phi_e=..., **metadata)
times, Vm, phi_e, meta = cc.load_result(path) # returns a 4-TUPLE (not a SimulationResult); phi_e/meta may be None/{}
# mesh I/O: cc.save_cardiac_mesh(path, data) / cc.load_cardiac_mesh(path) for a CardiacMeshData
```

10. Media — video & figures

The one-liner. No arguments beyond a name: full-frame, no labels, 1080p, standard colours.

```python
# runnable-video-section
import numpy as np, cardiac_core as cc
from cardiac_core import Video, Gradient, render

g    = cc.Grid(40, 10, 0.025)                                  # dx is REQUIRED (cm)
cond = cc.ConductivityConfig.bidomain(1.74, 6.25, chi=1400.0)
sim  = cc.monodomain(g, "ttp06", cond, cc.Stim.boundary(g, "left", amplitude=-80.0,
                                                        start_time=1.0, duration=2.0))
r    = sim.run(t_end=20.0, save_every=1.0)

info = r.video("my-wave", bulk=True)             # -> media/lab/_sim_outputs/videos/{date}/my-wave_01.mp4
print(info.backend, info.width, info.height, info.vmin, info.vmax)

# Colour is a reusable object. The RANGE is a scientific choice, not decoration:
# a 7.5 mV artifact spans 5.8% of the default -90..40 scale but 90.4% of the zoom window.
Gradient.physiological()        # -90..40 mV, viridis          (the default)
Gradient.rest_anchored()        # V_rest..40, inferno, grey masked tissue
Gradient.zoom(span=8.0)         # V_rest-0.3 .. V_rest+8       (make a small artifact visible)
Gradient.diverging()            # -90..50, RdBu_r
Gradient.autoscale()            # the data's own finite range
Gradient(cmap=["black", "red", "white"], gamma=1.4, levels=12)   # custom gradient

# Anything beyond the default is opt-in, on the Video spec or on render():
r.video("annotated", style="annotated", gradient=Gradient.rest_anchored(),
        isochrones=True, front=-40.0, speed=20.0, bulk=True)   # 20 ms of sim per real second

# Multi-panel: a panel IS a Video. Sharing one Gradient is what makes panels comparable.
render([Video.annotated(r, gradient=Gradient.physiological(), label="control"),
        Video.annotated(r, gradient=Gradient.physiological(), label="drug")],
       "control-vs-drug", question="lab", bulk=True, max_frames=20)

Video(r).preview(t_ms=10.0, bulk=True)           # ONE frame to PNG — check colours cheaply

cc.apd_map_figure(r, "my-wave", bulk=True)       # APD90 heatmap PNG
cc.activation_isochrones(r, "my-wave", bulk=True)  # activation-time contours PNG
cc.propagation_video(r, "my-wave", bulk=True)    # legacy one-liner (annotated, 600x300)
```

- `bulk=True` → gitignored `media/lab/_sim_outputs/...` (regenerable; the normal case). `bulk=False` → committed `media/{question}/...` (a curated figure worth keeping).
- `question=` names the owning research question; defaults to `"lab"`.
- `format=` is `"mp4"` (default), `"webm"` or `"gif"` — a GIF is filed under `images/` per the media convention. The encoder used is reported on `info.backend`; a fallback always warns.
- Figure-only knobs (`colorbar`, `title`, `figsize`/`dpi`, `label`, `front`, `isochrones`) RAISE on a bare clip rather than silently doing nothing — use `Video.annotated(...)`.
- `from cardiac_core.media import media_path` — raw path helper, if you save your own.

11. Full example — measure conduction velocity in a strip (the smoke pattern)

```python
import cardiac_core as cc

g    = cc.Grid(201, 51, 0.01)                                  # 2.0 × 0.5 cm strip (Nx=round(Lx/dx)+1)
cond = cc.ConductivityConfig.bidomain(1.74, 6.25, chi=1400.0)  # healthy ventricle σ
stim = cc.Stim.from_region(g, lambda x, y: x < 0.05, start_time=1.0, duration=2.0, amplitude=-80.0)

sim = cc.monodomain(g, "ttp06", cond, stim)
r   = sim.run(t_end=40.0, save_every=0.5)
cv  = r.cv(x1=20, x2=100, y=25)                                # ≈59 cm/s (dx=0.01, Crank-Nicolson/PCG)
print(f"conduction velocity = {cv:.1f} cm/s")
```

12. Compact end-to-end (the executable canary — new-feature smoke)

```python
# runnable-canary
import cardiac_core as cc
import os, tempfile

g    = cc.Grid(30, 8, 0.03)
cond = cc.ConductivityConfig.isotropic(1.4)
stim = cc.Stim.from_region(g, lambda x, y: x < 0.06, start_time=1.0, duration=2.0, amplitude=-52.0)

sim = cc.monodomain(g, "ttp06", cond, stim, dt=0.05)          # default pcg
r   = sim.run(t_end=10.0, save_every=1.0, record=("Vm", "ionic_states"))
cv  = r.cv(x1=5, x2=20, y=4)                                  # NaN until the front reaches x2 — call still valid
lat = r.lat()                                                 # (Nx, Ny) activation map
df  = cc.dominant_frequency(r.Vm, r.times, 5, 4)              # Hz at one node

# fast direct solve — valid on this uniform full-rectangle Neumann mesh:
fast = cc.monodomain(g, "ttp06", cond, stim, dt=0.05, linear_solver="dct").run(6.0, 2.0)

# drug block + inexcitable scar (each rebuilds from t=0):
sim.scale_conductance("GKr", 0.5)
scar = cc.rectangle_mask(30, 8, 0.03, 0.3, 0.0, 0.4, 0.24)
sim.set_conductivity(scar, D=0.0)
sim.run(6.0, 2.0)

# save / load (.npz): load_result returns a (times, Vm, phi_e, metadata) tuple
_p = os.path.join(tempfile.gettempdir(), "_cc_cheatsheet_demo.npz")
cc.save_result(_p, r.times, r.Vm)
t2, Vm2, phi2, meta2 = cc.load_result(_p)
os.remove(_p)
```